09. GPT 1

15 March 2026 13:13

Radford et al

NLU → wide range of tasks
- → text entailment
- → question answering
- → semantic similarity assessment
- → document classification
- → many others

unlabeled data → abundant
labeled data → scarce
↳ makes it a challenge for discriminatively trained models to perform adequately

↓

this can be improved by Generative Pre-training on large corpus of unlabeled text
↓
discriminative fine tuning on each specific task

Use task-aware input transformations during fine-tuning
to achieve effective transfer while requiring minimal changes to model architecture

Leveraging more than word level information from unlabeled text is challenging
↳ it is unclear what type of optimization objectives are most effective at learning text representations that are useful for transfer
↳ No consensus on most effective way to transfer the learned representations to a target task

this paper → Semi-supervised Approach → unsupervised pre-training + Supervised fine tuning
↓
to learn a universal representation that transfers with little adaptation to wide range of tasks

two stage training procedure → i use a language modelling objective on unlabeled data to learn the initial parameters of a neural network model
ii we adapt these parameters to a target task using corresponding supervised objective

We use Transformer

During transfer, we utilize task specific input adaptations derived from traversal style approaches
↓
which process structured text input as a single contiguous sequence of tokens
enable fine tuning with minimal architectural changes

Unsupervised pre training → goal is to find a good initialization point instead of modifying the supervised learning objective
acts as a regularization scheme, enabling better generalization in Neural networks

Auxiliary training objectives → Adding auxiliary unsupervised training objectives is an alternative form of semi supervised learning

3 Framework

two stages → i learning high capacity language model on a large corpus of text
ii fine tuning stage where we adapt the model to a discriminative task with labeled data

3.1 Unsupervised Pre-training

unsupervised corpus of tokens $\mathcal{U} = \{u_1, \dots, u_n\}$

Standard language modelling objective to maximize the following likelihood

$$L_1(\mathcal{U}) = \sum_i \log P(u_i \mid u_{i-k}, \dots, u_{i-1}; \Theta)$$

token sequence; current token; previous to new; K - size of context window

To succeed at this model must learn → syntax, semantics, co-reference, long dependence

model learns local conditional distributions

predict next token given prev tokens

Obj max log prob of correct token equivalent to minimizing cross entropy loss

P → conditional probability using a network with parameters Θ
↓
trained using SGD

They use a multi layer transformer decoder

Training pipeline
- Stage 1 Unsupervised language modelling (Learn parameters Θ)
- Stage 2 Supervised fine tuning (Adapt Θ for task)

Θ pretrained → contains linguistic structure
Θ fine tuned → specialized for task

first important design decision
Generative pre training
Language modelling objective
Model learns $P(w_i \mid w_1, \dots, w_{i-1})$ Goal Extract linguistic structure before seeing the task

constraints they are solving
High unlabeled text
Small labeled task data

the model must learn
grammar, syntax, semantics, long range dependencies
because predicting the next token requires them
representation emerges as a byproduct of prediction

finding an an optimization objective that transfers well is difficult
paper claims Language model is sufficient to learn transferable representations

Minimal architecture change during transfer
at that time existing methods
pretrained model → feature extractor → new task specific architecture
this paper does
single architecture + small task specific adaptation

Earlier semi supervised NLP methods
pretrained features → fixed → task model

Meaning
pretrained model freezes
GPT fine tuning does
pretrained model → continue training

Representation adapts to the task

Auxiliary training objectives
Idea Instead of one objective you add extra losses
eg Total loss = Task loss + POS tagging loss + Language modelling loss + Parsing loss
Purpose force the representation to encode linguistic structure
Auxiliary tasks act as regularizers on the representation space
They push the embedding space to encode syntax, semantics, structure
If auxiliary objectives are removed
Representation may overfit task specific signal
They always improve

unsupervised

local conditional distributions

→ conditional probability using a network with parameters Θ

↓

trained using SGD

They use a multi layer transformer decoder

↓

applies Multi head self-attention operation over input tokens followed by position wise FF layers

h_l = transformer block (h_{l-1}) input to the transformer ←

$$h_0 = UW_e + W_p$$
$$h_l = \text{transformer block}(h_{l-1}) \quad \forall l \in [1, n]$$
$$P(u) = \text{softmax}(h_n W_e^T)$$

$U = (u_{-k} \quad u_{-1})$ is the context vector of tokens

$n \rightarrow$ no of layers

$W_e \rightarrow$ token embedding matrix

$W_p \rightarrow$ position embedding matrix

↳ they use the embedding matrix for output generation

Weight tying (to reduce parameters & improve generalization)

3.7 Supervised fine tuning

After training the model with objective in above equation

we adapt the params to a supervised target task

← $C \rightarrow$ labeled dataset

each instance consists of seq of input tokens $x^1 \quad x^m$ along with a label y

Inputs passed through pretrained model → final transformer blocks activation h_l^m

↓

fed into an added linear output layer with parameters W_y to predict y

$$P(y|x^1 \quad x^m) = \text{softmax}(h_l^m W_y)$$

thus gives us the following objective to maximize

$$L_2(C) = \sum_{(x,y)} \log P(y|x^1 \quad x^m)$$

Model receives input sequence $x_1 \quad x_n$

label

process $x \rightarrow$ transformer $\rightarrow$ final hidden state

They use final token representation to make the prediction

Not mean pooling / attention pooling

Two gradients exist

$\nabla_\theta L_Task$

$\nabla_\theta L_language$

Total $\nabla_\theta (L_Task + L_language)$

They found including language modelling as an auxiliary objective to the fine tuning helped learning by (i) Improving generalization of the supervised model

(ii) Accelerating convergence

$$L_3(C) = L_2(C) + \lambda * L_1(C)$$

weight

the only extra params we require during fine tuning are W_y & embeddings for delimiter tokens

Meaning

Task loss + λ * Language modelling loss

During fine-tuning they continue language modelling

because without this representation drifts downstream task

with auxiliary LM → representation stays linguistically grounded

} Hence it acts as a regularizer

If auxiliary objectives are removed

Representation may overfit task specific signal

Thus paper proposes Do one powerful pretraining first → then fine tune

Unsupervised pre training doesn't only find good initialization → it enables learning representations containing linguistic structure

They believe model learns general linguistic structure not dataset specific patterns

↳ transferable language knowledge

Catastrophic forgetting → model already contains useful linguistic structure

fine tuning on a small dataset can overwrite it

Representation shift

pretrained space → task-specific space

too much shift destroys general knowledge

that is why fine tuning uses small learning rate

GPT — is an autoregressive transformer language model

Information flow view

tokens

↓

attention mixing

↓

contextual representations

↓

final token aggregates prefix information

↓

classifier reads final token

They introduce special tokens

START

DELIM

EXTRACT → used for prediction

↳ this tokens representation becomes the classification vector

the idea later evolved into BERT [CLS] token

3.3 Task specific input transformations

for certain tasks like text entailment that have structured inputs we require modifications

previous works → specific architectures

here → traversal style approach

↓

convert structured inputs into an ordered sequence that our pre-trained model can process

All transformations include randomly initialized start and end tokens (<s>, <e>)

for textual entailment → premise p hypothesis h delimiter token '$'

Similarity → no inherent order → modify the input seq to contain both sentences with a delimiter in between → process each independently to produce two sequence representations h_l^m which are added element wise before fed into linear layer

Q/A & commonsense reasoning →

context document z

question q

concatenate document context

question with each possible answer

Q/A & commonsense reasoning →

context document z

question q

set of possible answers $\{a_k\}$

concatenate document content question with each possible answer adding a delimiter token in bwn

$[z, q, \$, a_k]$

each of these sequences are processed independently with our model, normalized via Softmax to produce an output distribution over possible answers

Language modelling needs long range dependencies

GPT-1 architecture
- 12 transformer decoder layers
- hidden size = 768
- attention heads = 12
- ~ 117M parameters

GPT2 ~ 1.5B "

GPT3 ~ 175B "

Context length — 512 tokens

Why tried to test one hypothesis

Does LM pretraining produce transferable representations?

Language modelling captures syntactic knowledge

Limitations of GPT-1

1. Requires task specific fine tuning → Why limiting
   - ↳ for each task → train a new model
   - new task → new dataset → new fine tuning
   - Modern LLMs do single model → prompt → perform task
2. Small Model scale
   - ↳ simply lacked knowledge to store large amounts of knowledge
3. Limited context window
   - ↳ 512 tokens (modern models 128k+)
4. Autoregressive only
   - ↳ BERT solved it through MLM

Modern LLM training stages
1. Pretraining
2. Instruction tuning
3. RLHF

One pretrained transformer beats many task specific models

Transformer + Large scale unsupervised pretraining + End to end fine tuning

- Transformer → remove → long range reasoning weakens
- Large scale unsupervised pretraining → remove large scale corpus → representation quality collapses
- End to end fine tuning → removing it → frozen features

Analysis

Why does language model pretraining transfer so well?

Layer transfer
- ↳ how many pre trained layers should be reused?
- ↳ transfer diff no of transformer layers to the task
- → More layers transferred → better performance

each layer learns useful linguistic features

as whole stack contains transferable knowledge

Pretraining doesn't just learn embeddings & learns useful computation at every layer

Zero shot behavior (They convert tasks into probability distb^ns)
- ↳ test model without fine tuning (only language model)

Language modelling implicitly learns
- reasoning patterns
- semantic relations
- linguistic rules

} these can be extracted from probability distb^ns → foreshadows modern prompting

even without fine tuning model performs better above random baseline

Ablation studies

Removed Auxiliary LM objective
- ↳ performance slightly drops
- ↳ LM loss regularizes representation

Replaced transformer with LSTM

They conclude

A single model can perform many NLP tasks through Generative pretraining + Discriminative fine tuning

↳ performance slightly drops
↳ LM loss regularizes representation

Replaced transformer with LSTM
↳ performance drops
↳ transformer architecture transfers better
↓
better attention memory

Removed pretraining → provides most of performance gain
Train transformer from scratch on each task
performance drops

A single model can perform many NLP tasks through
Generative pretraining
+ Discriminative fine tuning

Key components → transformers
large unlabeled text
long context modelling